

Arzel Andrei Israel
Lynxie Jagonob
John Walter Pediangco
John Christian Setubal
Emmanuel Siastres

Lab 6

Lab 6: Building APIs with Python (The Flask Framework)

Objective: Recreate your API logic using Python and Flask.

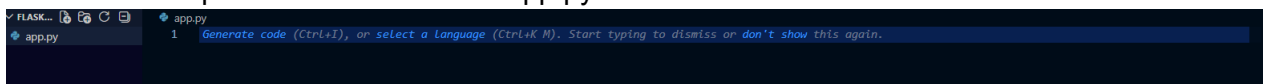
Setup Instructions:

1. Install Flask: You will need Python installed. In your terminal, create a new folder flask-api-lab and install Flask using pip (Python's version of npm).

- o Command: pip install flask

```
PS C:\Users\emman\OneDrive\Documents\Israel_Jagonob_Pediangco_Setubal_Siastres_lab_6\flask-api-lab> pip install flask
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Collecting werkzeug>=3.1.3 (from flask)
  Downloading werkzeug-3.1.3-py3-none-any.whl (226 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.1.3-py3-none-any.whl (116 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp313-cp313-win_amd64.whl (15 kB)
Installing collected packages: markupsafe, itsdangerous, colorama, blinker, werkzeug, jinja2, click, flask
Successfully installed blinker-1.9.0 click-8.4.1 colorama-0.4.6 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8
PS C:\Users\emman\OneDrive\Documents\Israel_Jagonob_Pediangco_Setubal_Siastres_lab_6\flask-api-lab>
```

2. Create the Script: Create a file named app.py.



Hints

In Flask, we don't use app.get or app.post like in Express. Instead, we use a single @app.route() decorator and specify the methods. Follow these hints to build your Python server:

- Initialization: * Import Flask, request, and jsonify from the flask module.
 - o Initialize your app using app = Flask(__name__).
- The GET Endpoint (/api/greeting):
 - o Use the @app.route('/api/greeting', methods=['GET']) decorator.
 - o Hint: Write a standard Python function directly under the decorator. This function should return jsonify({"message": "..."}).
- The POST Endpoint (/api/echo):
 - o Use @app.route('/api/echo', methods=['POST']).
 - o Data Capture: Inside the function, use request.get_json() to grab the incoming payload.

- Logic: Return that data back to the user using jsonify().
- The Run Command:
 - At the very bottom, use the standard Python if __name__ == "__main__": block and call app.run(port=5000).

```
app.py
1  from flask import Flask, request, jsonify
2
3  app = Flask(__name__)
4
5  @app.route('/api/greeting', methods=['GET'])
6  def greeting():
7      return jsonify({
8          "message": "Hello from Flask!"
9      })
10
11  @app.route('/api/echo', methods=['POST'])
12  def echo():
13      data = request.get_json()
14      return jsonify(data)
15
16  if __name__ == "__main__":
17      app.run(port=5000)
```

Testing with curl:

1. Start the Server: Run python app.py (or python3 app.py). Your server will likely run on Port 5000 (Flask's default).

```
PS C:\Users\emman\OneDrive\Documents\Israel_Jagonob_Pediangco_Setubal_Siastres_lab_6\flask-api-lab> python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

2. Test GET: Bash

curl <http://127.0.0.1:5000/api/greeting>

```
PS C:\Users\emman\OneDrive\Documents\Israel_Jagonob_Pediangco_Setubal_Siastres_lab_6\flask-api-lab> curl http://127.0.0.1:5000/api/greeting

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): Y

StatusCode      : 200
StatusDescription : OK
Content         : {"message":"Hello from Flask!"}

RawContent      : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 32
                  Content-Type: application/json
                  Date: Mon, 15 Jun 2026 17:01:22 GMT
                  Server: Werkzeug/3.1.8 Python/3.13.14

                  {"message":"Hello from Flask!"}

Forms           : {}
Headers         : {[Connection, close], [Content-Length, 32], [Content-Type, application/json], [Date, Mon, 15 Jun 2026 17:01:22 GMT]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 32
```

3. Test POST (The Payload Test): Bash curl -X

POST <http://127.0.0.1:5000/api/echo> \

-H "Content-Type: application/json" \

-d '{"framework": "Flask", "language": "Python"}'

The screenshot shows a web browser interface for testing HTTP requests. The URL bar shows <http://127.0.0.1:5000/api/echo>. The request method is set to POST. The request body is a JSON object:

```
{
  "framework": "Flask",
  "language": "Python"
}
```

. The response status is 200 OK, and the response body is the same JSON object. The browser interface includes tabs for Body, Cookies, Headers, Test Results, and a status bar at the bottom showing the response details.